Spec — Code Review Phase 0: Hardcoded Paths + Secrets

Created: 2026-06-19 — Claude.ai (updated with Grok feedback) Status: Design — starts when AWS Phase 0 (containerization) begins Runs: parallel to AWS Phase 0, not blocking it Reference: `docs/CODE_REVIEW_PLAN.md`

Context

Before Docker containers can be built, every hardcoded local path and every secret not driven by an environment variable must be found and fixed. This review runs alongside AWS Phase 0 containerization — findings directly unblock the Docker build. It is not a separate project; it is preparation that makes Phase 0 go smoothly.

Stay ruthless about scope: only review what blocks containerization. Everything else is Phase 1-4 work.

App review order

Order	App	Why
1st	`minimoi_portal/`	Entry point — auth, routing, shared proxy. Security issues here affect everything.
2nd	`domains/german/`	Most active development, highest churn, most likely to have new issues.
3rd	`curator/`	More stable, lower risk for this specific review.

How the review actually runs

This is an AI-assisted review, not a manual audit. The workflow:

1. **Grok or Claude Code** receives specific instructions + file paths for each app in order
2. **Reviewer** reads the files and produces a findings list using the grep commands below — no guessing, actual code search
3. **Robert confirms** findings before any fix is applied
4. **Claude Code** applies the fixes

5. **Findings report** documents what was found vs. what was fixed

Each app is one review session. Three sessions total for Phase 0.

Review 1 — Hardcoded paths

What to find

Every path that will not exist in a Docker container or on EC2:

```python
# Find these:
open('/Users/vanstedum/...')
os.path.expanduser('~/Projects/...')
Path('/Users/vanstedum/...')
```

Search commands

```bash
grep -rn "vanstedum" . --include="*.py"
grep -rn "expanduser" . --include="*.py"
grep -rn "~/Projects" . --include="*.py" --include="*.json"
grep -rn "os.path.join.*home" . --include="*.py"
# Any open() not using a relative path or env var:
grep -rn "open(" . --include="*.py" | grep -v "environ\|DATA_DIR\|\\./"
```

Also check: launchd plist files, shell scripts, config JSON files.

Fix pattern

```python
# Before
open('/Users/vanstedum/Projects/personal-ai-agents/data/cos_context.json')

# After
DATA_DIR = os.environ.get('DATA_DIR', './data')
open(os.path.join(DATA_DIR, 'cos_context.json'))
```

Review 2 — Secrets and credentials

What to find

Any secret, API key, or credential not loaded from an environment variable. Also: macOS Keychain calls that won't work on Linux EC2.

Search commands

bash

```bash
grep -rn "sk-" . --include="*.py" --include="*.json"
grep -rn "xai-" . --include="*.py"
grep -rn "api_key\s*=" . --include="*.py" | grep -v "environ\|os.get"
grep -rn "password\s*=" . --include="*.py" | grep -v "environ"
grep -rn "keyring\|security find-generic-password" . --include="*.py"
# Check for secrets in log statements:
grep -rn "logger\.\(info\|debug\|print\).*key\|token\|secret" . --include="*.py
```

Fix pattern

python

```python
# Before
OPENAI_API_KEY = "sk-abc123"

# After
OPENAI_API_KEY = os.environ['OPENAI_API_KEY']
```

Review 3 — .env.example completeness

After fixes, confirm `.env.example` documents every env var in code:

bash

```
# Find all env var references in code
grep -rh "os\.environ" . --include="*.py" | \
  grep -oP "(?<=\[')[^']*(?='\])|(?<=get\(')[^']*(?=')" | \
  sort -u > /tmp/code_vars.txt

# Compare against .env.example keys
grep -v "^#\|^$" .env.example | cut -d= -f1 | sort > /tmp/example_vars.txt

# Show vars in code but missing from .env.example
diff /tmp/example_vars.txt /tmp/code_vars.txt
```

Findings report format

Produce `docs/code-review/findings_phase0_YYYY-MM-DD.md`:

```
markdown
```

```
# Code Review Findings — Phase 0
*Date · Reviewer · App reviewed*

## Hardcoded paths

|
 File
|
 Line
|
 Issue
|
 Severity
|
 Effort
|
 Fix applied
|
|
------
|
------
|
-------
|
----------
|
--------
|
-------------
|
|
 portal/app.py
|
 47
|
 /Users/vanstedum/...
|
 Blocking
|
 5 min
|
 ✓ → DATA_DIR
```

```
|

## Secrets

|
 File
|
 Line
|
 Issue
|
 Severity
|
 Effort
|
 Fix applied
|
|
------
|
------
|
-------
|
----------
|
--------
|
-------------
|

## .env.example gaps

|
 Variable
|
 Found in
|
 Severity
|
 Added
|
|
----------
```

```
|
---------
|
----------
|
-------
|

## Summary
- Blocking issues: X found, X fixed, X remaining
- High issues: X found, X fixed
- Medium issues: X found, X fixed
- docker build status after fixes: pass / fail
```

Severity definitions:

- **Blocking** — prevents Docker build or exposes credentials
- **High** — security risk or will fail silently on EC2
- **Medium** — should fix but won't block Phase 0

Definition of Done

- All three apps reviewed in order (Portal → German → Curator)
- All Blocking findings fixed before `docker build` attempt
- No hardcoded local paths anywhere in the codebase
- No secrets in code — all loaded from `os.environ`
- macOS Keychain calls removed or abstracted
- `.env.example` complete — every env var in code documented
- `.env` confirmed in `.gitignore`
- `docker build` succeeds without path-related errors
- Findings report committed to `docs/code-review/`
- Significant findings produce a Decision Record

Commit

```
Code review Phase 0: replace hardcoded paths and secrets with env vars.
Findings: docs/code-review/findings_phase0_YYYY-MM-DD.md
```

Spec · 2026-06-19 · Claude.ai Parallel track to AWS Phase 0 — not a blocker, an enabler Next: spec_code_review_phase1_auth_errors.md (after Phase 0 complete)